SeeBus - מדריך למתכנת, התקנה ופריסה

סקירה ארכיטקטונית

SeeBus הוא יישום Web במבנה Monorepo. ה־Frontend מציג זמני הגעה ומספק מסך ניהול. ה־Backend מספק REST API, שומר תצורה ונתוני תפוסה של התחנות בזיכרון ומחשב לוח הגעות במחזוריות.

```
    Users / Administrators
              |
              v
    React + TypeScript + Vite
      Public view / Admin UI
              |
         HTTPS REST API
              |
              v
     Java 21 + Spring Boot
     +--------------------+
     | Configuration API  |
     | Readings API       |
     | Scheduling engine  |
     | Token security     |
     +--------------------+
              |
         In-memory state
```

```
Production hosting:
  Frontend -> Cloudflare Workers (static assets)
  Backend  -> Docker Compose on a server -> HTTPS gateway / tunnel
(configured separately)
```

הערה: אין בפרויקט מסד נתונים. אתחול תהליך השרת מאפס תחנות, אוטובוסים, לוח בסיסי והיסטוריית מדידות. חשוב להביא זאת בחשבון בעת פריסה ותחזוקה.

טכנולוגיות ומבנה הפרויקט

תחום	טכנולוגיה וקובץ מרכזי
Frontend	React 19, TypeScript, Vite 8, Chakra UI 3, React Router 7
Backend	Java 21, Spring Boot 4.1.1, Spring MVC/Security/Validation, Maven
API	HTTP + JSON, Spring Controllers backend/src/main/java/com/seebus/backend/api
Backend Deployment	Dockerfile + Docker Compose backend/Dockerfile deploy/docker-compose.yml
Frontend Deployment	Cloudflare Wrangler frontend/wrangler.jsonc
Scripts	PowerShell

```
seebus/
├── backend/
│   ├── src/main/java/com/seebus/backend/
│   │   ├── api/          REST controllers
│   │   ├── security/     Token authentication and CORS
│   │   ├── config/       Configuration validation
│   │   ├── station/      Stations
│   │   ├── bus/          Buses
│   │   ├── reading/      Occupancy readings
│   │   └── schedule/     Scheduler and policies
│   ├── src/main/resources/application.properties
│   ├── Dockerfile
│   └── pom.xml
├── frontend/
│   ├── src/api/seebusApi.ts
│   ├── src/pages/
│   ├── src/components/admin/tabs/
│   ├── src/types/
```

```
│   ├── package.json
│   ├── vite.config.ts
│   └── wrangler.jsonc
├── deploy/docker-compose.yml
├── scripts/
└── .env.example
```

סביבת פיתוח מקומית

דרישות מקדימות

- *JDK 21*, *Node.js*, *npm* בגרסאות התואמות ל-*Vite 8*, *PowerShell*.
- חיבור אינטרנט לצורך התקנת תלויות *Maven* ו־*npm*.
- *Docker* עם רכיב *Compose* וחשבון *Cloudflare* לצורך פריסת אתר ה-*Frontend*.

הרצה מהירה ב־Windows PowerShell

מהתיקייה הראשית של הפרויקט:

```
cd frontend
npm ci
cd ..
.\scripts\dev.ps1
```

אם רוצים להחליף ערכי פיתוח בקובץ מקומי:

```
.\scripts\setup.ps1   # Creates .env from .env.example
# Edit .env with local values
.\scripts\dev.ps1 -UseEnv
```

ערכי פיתוח אפשריים: (ניתן בהדרגה להריץ חלקים לוקלית וחלקים על למשל שרת בענן)

שירות	כתובת ברירת מחדל בפיתוח (לא בפרודקשן!)
אתר	http://localhost:5173
REST API	http://localhost:8080
Health	http://localhost:8080/api/health
Swagger UI	http://localhost:8080/swagger-ui.html
OpenAPI	http://localhost:8080/v3/api-docs

Swagger זמין כאשר *SEEBUS_SWAGGER_ENABLED = true*

אפשר להריץ בנפרד את ה-*backend* ואת ה-*frontend* אם יש צורך (הסקריפט *dev* מריץ את שניהם באותו הטרמינל)

```
cd backend
.\mvnw.cmd spring-boot:run
```

```
cd frontend
npm run dev
```

משתני סביבה

משתנה	שימוש
SEEBUS_ADMIN_TOKEN	טוקן גישה לפעולות ניהול בשרת
SEEBUS_SENSOR_TOKEN	טוקן גישה ל-*POST /api/readings*
SEEBUS_SWAGGER_ENABLED	הפעלת *Swagger / OpenAPI* ברירת המחדל בשרת היא *false*
SEEBUS_API_URL	כתובת שרת *API* לסקריפטים המקומיים
VITE_API_BASE_URL	כתובת בסיס שה-*Frontend* משתמש בה מוזרקת בזמן בנייה וגלויה בדפדפן
SEEBUS_DEPLOY_HOST	יעד *SSH* עבור *scripts/deploy.ps1*

אין להכניס סודות למשתני VITE_*, או לשמור את .env ב־Git. בפריסה אמיתית יש להשתמש בטוקנים חזקים ושונים מאלו המופיעים בקובץ הדוגמה.

Backend - מודל, שירותים ותזמון

מודל הנתונים

- $Station(id, name)$ = תחנה
- $Bus(id, name, capacity)$ = אוטובוס וקיבולת מוגדרת
- $ScheduleEntry(busId, stationId, time)$ = הגעה בסיסית בשעה מקומית
- $StationReading(id, stationId, occupancy, measuredAt, receivedAt)$ = מדידת תפוסה בתחנה בין 0 ל – 1
- $ScheduledStop(busId, stationId, arrivalTime)$ = אירוע הגעה מחושב
- $CurrentSchedule(generatedAt, policy, entries)$ = לוח מחושב המוחזר לציבור
- $SchedulingInput(\dots)$ = צילום מצב המועבר למדיניות התזמון

השירותים המרכזיים

רכיב	אחריות
$ConfigService$	אימות ועדכון תחנות, אוטובוסים ולוח בסיסי במשותף
$StationService,\ BusService$	שמירת הישויות בזיכרון
$ScheduleService$	אחסון רשומות הלוח הבסיסי
$ReadingService$	אימות וקבלת מדידות, האחרונה לכל תחנה והיסטוריה של עד 100 קריאות
$DefaultScheduleResolver$	חישוב מופע ההגעה הקרוב מכל שעת בסיס
$SchedulingPolicyService$	רשימת מדיניות ובחירת מדיניות פעילה
$SchedulingService$	חישוב לוח עדכני והחזקת התוצאה
$SchedulerLoop$	הרצת חישוב בערך אחת ל־5 שניות

מדיניות התזמון הקיימות

$default$ - מחזירה את מופע ההגעה הקרוב היום או מחר

$sensor-priority$ -בודקת מדידת תפוסה של לפחות 50% שהתקבלה ב־30 השניות האחרונות, ומקדימה את ההגעה הקרובה לתחנה לזמן של $now\ +\ 1\ minute$. התחייבויות זמניות מונעות יצירת הקדמה חדשה בכל חישוב.

להרחבת המנוע ניתן להוסיף *@Component* המממש *SchedulingPolicy*.
יש צורך להחזיר שם ייחודי מ־*getName()* ולחשב *CurrentSchedule* על בסיס *SchedulingInput*.

המדיניות תזוהה אוטומטית ברשימת האפשרויות.

אחסון, עקביות ומגבלות

- התחנות והאוטובוסים נשמרים ב-*ConcurrentHashMap*, נתוני המדידות נשמרים בהיסטוריה מסונכרנת ובמפה של הקריאה האחרונה לכל תחנה
- הלוח הנוכחי מוחזק ב-*AtomicReference* כדי לאפשר קריאה עקבית של תוצאת חישוב אחת
- *PUT /api/admin/config* מאמת תצורה שלמה, אך החלפת הנתונים במספר שירותים אינה טרנזקציה אטומית. לפני הרחבה לעומסים או לכותבים רבים יש לטפל בעקביות ובנעילות.
- אין שמירה לדיסק, גיבוי מובנה או ניהול משתמשים מלא.

חוזי REST API ואבטחה

Method + Path	הרשאה	שימוש
GET /api/health	PUBLIC	בדיקת זמינות
GET /api/schedule/current	PUBLIC	לוח ההגעות המחושב
GET /api/admin/verify	ADMIN	אימות טוקן מנהל
GET /api/admin/config	ADMIN	קבלת תצורה מלאה
PUT /api/admin/config	ADMIN	החלפת תחנות, אוטובוסים ולוח בסיסי
GET /api/admin/policy	ADMIN	מדיניות פעילה ואפשרויות
PUT /api/admin/policy	ADMIN	בחירת מדיניות
GET /api/stations	ADMIN	קריאת תחנות
POST /api/stations	ADMIN	יצירת תחנה
GET /api/buses	ADMIN	קריאת אוטובוסים
GET /api/schedule/default	ADMIN	לוח בסיסי
POST /api/readings	SENSOR	קבלת מדידת תפוסה
GET /api/readings	ADMIN	היסטוריית מדידות
GET /api/readings/latest	ADMIN	קריאה אחרונה לכל תחנה

דוגמת בקשה לשמירת תצורה מלאה:

```
PUT /api/admin/config HTTP/1.1
Authorization: Bearer <ADMIN_TOKEN>
Content-Type: application/json

{
  "stations":   "id":"station-1","name":"Central Station"  ,
  "buses":   "id":"bus-1","name":"Line 1","capacity":40  ,
  "defaultSchedule":
```

```
    "busId":"bus-1","stationId":"station-1","time":"08:30"

}
```

SecurityConfig מאפשר גישה ציבורית ל-*Health* וללוח נסיעות נוכחי, ומגן על ניהול ועל נתוני מדידות באמצעות *Bearer tokens*.
מדיניות CORS מאפשרת כרגע את אתר *SeeBus* הראשי, כתובות *Preview* המתאימות לתבנית מוגדרת של *Cloudflare* ואת *http* : *// localhost* : 5173. אם שם המתחם משתנה, יש לעדכן את רשימת המקורות בשרת ולפרוס מחדש. *Swagger* נרשם כנתיב מותר, אך יצירתו מותנית במשתנה ההפעלה.

מסכים וזרימת נתונים Frontend

קובץ	תפקיד
src / App. tsx	*Routing*
src / pages / SchedulePage. tsx	בקשת לוח עדכני כל 5 שניות והצגת זמנים
src / pages / AdminPage. tsx	מעבר בין ממשק הסיסמה למסך הניהול
src / components / admin / AdminGate. tsx	בדיקת טוקן ושמירה ב-*sessionStorage*
src / components / admin / AdminDashboard. tsx	עריכת תצורה בזיכרון הממשק ושמירה בעת לחיצה
src / components / admin / tabs /	לשוניות *Policies*, *Schedule*, *Buses*, *Stations*, *Readings*
src / api / seebusApi. ts	פונקציות *Fetch* לכל קריאות ה-*API*
src / types	טיפוסי *TypeScript* של מודל הנתונים

פריסה (Deployment)

הפרויקט מגדיר שתי פריסות נפרדות: Backend במכולת Docker על שרת, ו־Frontend ב־Cloudflare Workers עם נכסים סטטיים.

Backend - הכנת שרת

בשרת Linux מותקנים Git, Docker ו־Docker Compose. העלו את המאגר לשרת ועברו לתיקיית השורש שלו. צרו קובץ *deploy / . env* מתוך *deploy / . env. example* והגדירו בו טוקנים לזיהוי האדמין והסנסורים:

```
SEEBUS_SENSOR_TOKEN=<long-random-sensor-token>
SEEBUS_ADMIN_TOKEN=<long-random-admin-token>
```

Backend - בנייה והפעלה

מתוך תיקיית השורש של המאגר, על שרת Linux:

```
# Initial deployment or after a code update
# Run 'git pull --ff-only' only when the working tree is clean.
git pull --ff-only

docker compose --env-file deploy/.env \
  -f deploy/docker-compose.yml up -d --build
```

```
# Status and logs
docker compose --env-file deploy/.env \
  -f deploy/docker-compose.yml ps
docker compose --env-file deploy/.env \
  -f deploy/docker-compose.yml logs --tail=100 backend

# Check from the server itself
curl http://127.0.0.1:8080/api/health
```

להפסקת השירות:

```
docker compose --env-file deploy/.env \
  -f deploy/docker-compose.yml down
```

חשיפת ה־Backend ב־HTTPS

ה־Compose מאזין ב־localhost ולכן אינו מפרסם ישירות את ה־API לאינטרנט. נדרש Reverse Proxy או Tunnel שמנתב כתובת HTTPS ציבורית אל http://127.0.0.1:8080 בשרת. לדוגמה, אם הוגדר אצלכם שם מתחם ייעודי ל־API, יש לוודא ש־https://<api-domain>/api/health מחזיר הצלחה מבחוץ.

אין לחשוף פורט ניהול ישירות ללא HTTPS. אם משנים כתובת Frontend, יש לעדכן גם את Allowed Origins ב־SecurityConfig.java ולבנות מחדש את ה־Backend.

Frontend - בנייה ופריסה ל־Cloudflare

1. ודאו שיש לכם חשבון Cloudflare וש־Wrangler מורשה לפעול מולו (למשל npx wrangler login).
2. הגדירו ל־Frontend לפני הבנייה את VITE_API_BASE_URL ככתובת ה־HTTPS החיצונית של ה־Backend, ללא /api בסוף.
3. בצעו Build ו־Deploy מתוך frontend/

```
cd frontend
npm ci

# Example: set the real public API endpoint for this build
$env:VITE_API_BASE_URL = "https://<api-domain>"

npm run build
npm run deploy
```

בדיקות לאחר פריסה

1. פתחו בדפדפן את כתובת ה־Frontend וודאו שעמוד / עולה
2. בדקו GET https://<api-domain>/api/health
3. פתחו /admin, הזינו טוקן מנהל ובדקו טעינת תצורה
4. צרו תחנה, אוטובוס ורשומת לוח, שמרו וחזרו לעמוד הציבורי
5. בדקו ש־GET /api/schedule/current/ מחזיר נתונים ושאין שגיאות CORS בכלי המפתחים של הדפדפן

בדיקות, ניטור ותחזוקה

```
# Backend tests
cd backend
.\mvnw.cmd test

# Frontend checks
cd ..\frontend
npm run build
npm run lint
```

תקלה	גורם אפשרי / טיפול
אתר נטען ללא נתונים	שגוי *API URL* כבוי *Backend* בעיית *CORS*
401 / 403	טוקן *Bearer* חסר או אינו מתאים להרשאה
בשמירת תצורה 400	מזהים כפולים, ערכים לא תקינים או הפניה לישות חסרה
שינויי הגדרות נעלמו	השרת הופעל מחדש, ואין כרגע DB
Backend עובד מקומית אבל לא מבחוץ	Tunnel/Proxy/DNS/HTTPS לא הוגדרו או אינם פעילים
npm run deploy נכשל	Wrangler התחברות הגדרת חשבון כושל Build פרויקט או